

Sistema de Recomendação de Cursos

Integrantes:

-Lucca Borges RM554608

-Ruan Vieira RM557599

-Rodrigo Carnevale RM55814

Sumário

1. Formulação do Problema	3
2. Visão Geral das Funções	3
2.1 Função mostrar(texto)	3
2.2 Função montar_lista_cursos(area_foco)	3
2.3 Função coletar_perfil_usuario()	4
2.4 Função calcular_relevancia_cursos(cursos, area_foco, prioridade)	4
2.5 Função merge_sort_lista(lista, chave, memo)	5
2.6 Função montar_plano_otimo(horas_max, orcamento_max, area_foco, prioridade)	5
2.7 Função executar_sistema()	6
3. Recursão e Memoização na Solução	7
4. Estrutura de Saída e Relatórios	7

1. Formulação do Problema

O objetivo do sistema é montar um plano de estudos em IA para um profissional, respeitando restrições de tempo disponível (horas de estudo) e orçamento máximo, maximizando a relevância total dos cursos escolhidos.

Entradas principais:

- Área de atuação do profissional (saúde, direito, educação, engenharia, administração ou genérico).
- Prioridade dada ao tema IA (escala de 1 a 10).
- Estilo de estudo (curto e barato, equilibrado ou aprofundado), que define horas e orçamento.

Saídas principais:

- Catálogo de cursos personalizados com relevância calculada.
- Trilha de cursos selecionados pela mochila 0/1 (plano ótimo).
- Resumo em formato de relatório: horas usadas, orçamento gasto e relevância total.

2. Visão Geral das Funções

A seguir, cada função do código é explicada em detalhes, incluindo seu papel na solução, parâmetros de entrada, retornos e como se relaciona com recursão, memoização e algoritmo da mochila.

2.1 Função `mostrar(texto)`

A função `mostrar(texto)` é uma função auxiliar de interface para o Jupyter Notebook. Ela recebe uma string e utiliza `IPython.display.HTML` para exibir o texto formatado dentro de uma tag `<pre>`, o que melhora a legibilidade dos menus e relatórios apresentados ao usuário.

Ela não é recursiva, pois seu papel é apenas de saída/visualização, sem lógica algorítmica.

2.2 Função `montar_lista_cursos(area_foco)`

Responsável por construir o catálogo dinâmico de cursos. A função recebe a área de foco escolhida pelo usuário (por exemplo, “saúde”, “direito”, “engenharia”) e retorna uma lista de dicionários, onde cada dicionário representa um curso com os campos:

- `curso` – nome do curso;
- `area` – área temática;

- horas – carga horária;
- preco – valor do curso;
- impacto – impacto base (1 a 10).

A função começa com uma lista fixa de cursos gerais e utiliza um dicionário com pelo menos cinco cursos específicos para cada área profissional (saúde, direito, educação, engenharia e administração).

Caso o usuário escolha uma área específica, a função:

1. Adiciona todos os cursos dessa área;
2. Adiciona também três cursos de cada uma das outras áreas, garantindo variedade e mantendo coerência com o catálogo ampliado.

Se a área informada não for reconhecida (área “genérica”), a função inclui todos os cursos de todas as áreas, gerando um catálogo multidisciplinar completo.

Com esse mecanismo, o catálogo final sempre possui 20 ou mais cursos, atendendo diretamente ao requisito do enunciado para trabalhar com pelo menos vinte informações diferentes.

2.3 Função `coletar_perfil_usuario()`

Esta função cuida da interação inicial com o usuário e coleta os parâmetros necessários para o problema de mochila. Ela apresenta menus no console (via `mostrar`) para:

- Selecionar a área profissional (1 a 6);
- Informar a prioridade de IA (1 a 10);
- Escolher o estilo de estudo (curto e barato, equilibrado ou aprofundado).

Com base no estilo de estudo, a função retorna uma tupla com:

- `area_foco`: string da área mapeada (por exemplo, 'saúde');
- `prioridade`: inteiro entre 1 e 10;
- `horas_max`: horas máximas de estudo (por exemplo, 8, 15 ou 25);
- `orcamento_max`: valor máximo disponível para investimento em cursos (por exemplo, 800, 1500 ou 3000).

Esses valores são exatamente as restrições que alimentam o algoritmo da mochila.

2.4 Função `calcular_relevancia_cursos(cursos, area_foco, prioridade)`

Recebe a lista de cursos e calcula um novo atributo chamado 'relevancia' para cada curso. A lógica usada é:

1. Começa do valor de 'impacto' definido manualmente para o curso (nota de 1 a 10);
2. Soma um bônus de área: +2 se o curso for da mesma área do profissional, +1 caso

contrário;

3. Soma um extra baseado na prioridade (prioridade // 5), que vale 0, 1 ou 2 dependendo da importância de IA.

No final, a relevância é limitada a no máximo 10. A função retorna uma nova lista de dicionários, cada um agora contendo o campo 'relevancia'. Essa relevância será usada tanto na ordenação (merge sort) quanto na função de valor do problema de mochila.

2.5 Função `merge_sort_lista(lista, chave, memo)`

Implementa o algoritmo de ordenação Merge Sort de forma recursiva com memoização manual. Seu objetivo é ordenar a lista de cursos de acordo com uma chave fornecida (por exemplo, relevância por hora).

Parâmetros:

- lista: lista de objetos (no caso, dicionários representando cursos).
- chave: função que recebe um item e devolve o valor usado para comparação na ordenação.
- memo: dicionário usado para memoização, onde a chave é uma tupla com os IDs dos elementos da lista.

A função funciona da seguinte forma:

1. Gera uma tupla com `id(x)` para cada elemento `x` da lista e verifica se essa tupla já está em `memo`. Se estiver, retorna imediatamente a versão ordenada que foi memorizada.
2. Caso a lista tenha tamanho 0 ou 1 (caso base da recursão), ela já está ordenada. Nesse caso, armazena a lista em `memo` e retorna.
3. Para listas maiores, a função encontra o meio, divide a lista em duas metades e chama `merge_sort_lista` recursivamente para cada metade.
4. Depois, intercalam-se os dois vetores ordenados (esquerda e direita) comparando a `chave(e[i])` e `chave(d[j])` até consumir todos os elementos. O resultado final também é armazenado em `memo`.

Dessa forma, `merge_sort_lista` usa recursão (chamando a si mesma em sublistas) e memoização para evitar retrabalho em sublistas que já foram ordenadas anteriormente.

2.6 Função `montar_plano_otimo(horas_max, orcamento_max, area_foco, prioridade)`

É a função central de lógica de negócio, responsável por aplicar o algoritmo da mochila 0/1. Ela integra as etapas de geração do catálogo, cálculo de relevância, ordenação via merge sort e, por fim, a otimização via programação dinâmica com recursão e memoização.

Passos principais:

1. Chama `montar_lista_cursos(area_foco)` para obter o catálogo inicial.
2. Chama `calcular_relevancia_cursos(...)` para adicionar o campo 'relevancia' a cada curso.

3. Cria um dicionário `memo_sort` e chama `merge_sort_lista(...)` para ordenar os cursos usando a chave `relevancia/horas`, priorizando cursos com maior relevância relativa por hora.

4. Define internamente a função recursiva `knapsack(i, h, o)`, decorada com `@lru_cache(maxsize=None)`, que implementa a mochila 0/1:

- `i`: índice do curso que está sendo considerado;
- `h`: horas restantes disponível para alocação;
- `o`: orçamento restante.

A função `knapsack` segue a lógica:

- Caso base: se `i` chegou ao final da lista ou se `h == 0` ou `o == 0`, não é possível adicionar mais cursos, logo o retorno é 0.
- Caso recursivo: primeiro calcula o valor 'melhor' sem pegar o curso atual (chamando `knapsack(i+1, h, o)`).
- Em seguida, verifica se o curso cabe nas restrições de horas e orçamento. Se couber, calcula o valor alternativo de pegar o curso, que é `relevancia_do_curso + knapsack(i+1, h - horas_curso, o - preco_curso)`. O melhor dos dois (pegar ou não pegar) é retornado.

O uso de `@lru_cache` garante a memoização automática, armazenando resultados de combinações de `(i, h, o)`, evitando recomputar estados repetidos.

5. Após obter o valor máximo possível com `knapsack(0, horas_max, orcamento_max)`, a função reconstrói a solução percorrendo os índices dos cursos. Para cada índice `i`, ela compara `knapsack(i, h, o)` com `knapsack(i+1, h, o)`. Se os valores forem diferentes, significa que o curso `i` foi escolhido na solução ótima. Nesse caso, o curso é adicionado à lista 'escolhidos' e as variáveis `h` e `o` são atualizadas, subtraindo as horas e o preço do curso escolhido.

6. Ao final, a função retorna quatro coisas importantes:

- Um `DataFrame` com o catálogo completo (`cursos_ordenados`);
- Um `DataFrame` com os cursos selecionados na trilha ótima (`escolhidos`);
- O valor máximo de relevância total obtido pela mochila;
- As horas e o orçamento efetivamente utilizados.

2.7 Função `executar_sistema()`

A função `executar_sistema()` é o ponto de entrada da aplicação. Ela implementa um menu simples de loop que permite ao usuário montar quantos planos de estudo quiser, até escolher a opção de sair.

Dentro do laço `while True`, a função:

- Exibe o menu principal usando `mostrar(...)`;
- Lê a opção digitada pelo usuário (0 ou 1);
- Se a opção for '0', exibe a mensagem de saída e interrompe o laço;
- Se a opção for '1', chama `coletar_perfil_usuario()` para obter os parâmetros do problema e, em seguida, `montar_plano_otimo(...)`, recebendo o catálogo completo, a trilha selecionada e o resumo numérico.

Depois, exibe:

- O catálogo personalizado (`df_cat`) com colunas `curso`, `area`, `horas`, `preco`, `relevancia`;
- A melhor trilha (`df_sel`), se houver cursos selecionados;
- Um resumo textual com horas usadas, orçamento gasto e relevância total.

3. Recursão e Memoização na Solução

A solução utiliza recursão e memoização em dois pontos principais: no algoritmo de ordenação (`merge sort`) e no algoritmo da mochila 0/1 (função `knapsack`).

- No `merge_sort_lista`, a recursão é usada para dividir a lista em sublistas menores até chegar ao caso base de tamanho 0 ou 1, e depois combinar os resultados. A memoização é feita manualmente por meio do dicionário `memo`, usando como chave uma tupla com os IDs dos elementos da lista original.

- Na função `knapsack`, definida dentro de `montar_plano_otimo`, a recursão mapeia o espaço de estados do problema de mochila, onde cada estado é representado por (i, h, o) . A memoização é implementada pelo decorador `@lru_cache`, que armazena os resultados de chamadas já realizadas com a mesma combinação de parâmetros. Isso garante eficiência, transformando uma solução exponencial em uma solução pseudo-polinomial típica de programação dinâmica.

4. Estrutura de Saída e Relatórios

A saída do sistema é organizada de forma a facilitar a análise do professor/usuário:

- Tabelas (`DataFrames`) com o catálogo completo de cursos, incluindo relevância calculada;
- Tabela apenas com os cursos selecionados pelo algoritmo da mochila, mostrando claramente quais cursos compõem a trilha recomendada;
- Um resumo textual com horas usadas, orçamento gasto e relevância total, permitindo avaliar rapidamente se o plano está aproveitando bem os recursos disponíveis.

Essa estrutura atende ao requisito do enunciado de apresentação de resultados e relatórios, mostrando de forma organizada as decisões tomadas pelo algoritmo.